Transformers

План

- NLP, работа с последовательностями
- Обзор более ранних архитектур
- Attention is all you need
- Особенности тренировки
- Vision Transformer (ViT)

https://jalammar.github.io/illustrated-transformer/

Обработка естественного языка (Natural Language Processing, NLP)

Примеры задач по работе с последовательностями данных:

- Распознавание речи (speech recognition)
- Понимание естественного языка (natural language understanding)
- Генерация естественного языка (natural language generation)

Задачи при работе с текстом

1. Классификация текстов (Выставление оценок, Эмоциональная окраска)
2. Перевод текстов
3. Распознавание именованных сущностей (классификация слов на части речи или выделение имен людей, компаний, даты и тд)
4. Продолжение последовательностей (Дать ответ продолжить предложение)

Контекстные векторы слов

Идея определять смысл слова на основе его контекста

1) фиксируем размер словаря, 50К
2) строим матрицу слова*слова
3) окном длины 5 определяем контекст

	a	horse	ride	bicycle	frame	rose
a		256	45	230	90	134
horse	256		137	4	2	5
ride	45	137		120	34	3
bicycle	230	4	120		76	2
frame	90	2	34	76		0
rose	134	5	3	2	0	

https://www.youtube.com/watch?v=WbtQzAvhnR

Контекстные векторы слов

Плюсы:

- Можем сравнивать вектора по MSE или cos distance

Недостатки:

- Векторы разрежены и занимают много памяти
- Ограничены словарем (слова не попавшие в него не обрабатываются)
- При изменении словаря нужно заново пересчитывать векторы
- Векторы редких слов не очень информативны

Контекстные векторы слов

a		256	45	230	90	134
horse	256		137	4	2	5
ride	45	137		120	34	3
bicycle	230	4	120		76	2
frame	90	2	34	76		0
rose	134	5	3	2	0	

50.000

a	45	56	41	134	-67	14
horse	25	32	10	67	1	0
ride	43	-17	-4	-10	49	32
bicycle	23	21	10	56	6	-2
frame	9	2	34	6	45	0
rose	54	59	-31	12	-45	9

500

метод понижения размерности
(PCA, TSNE)

Word2Vec

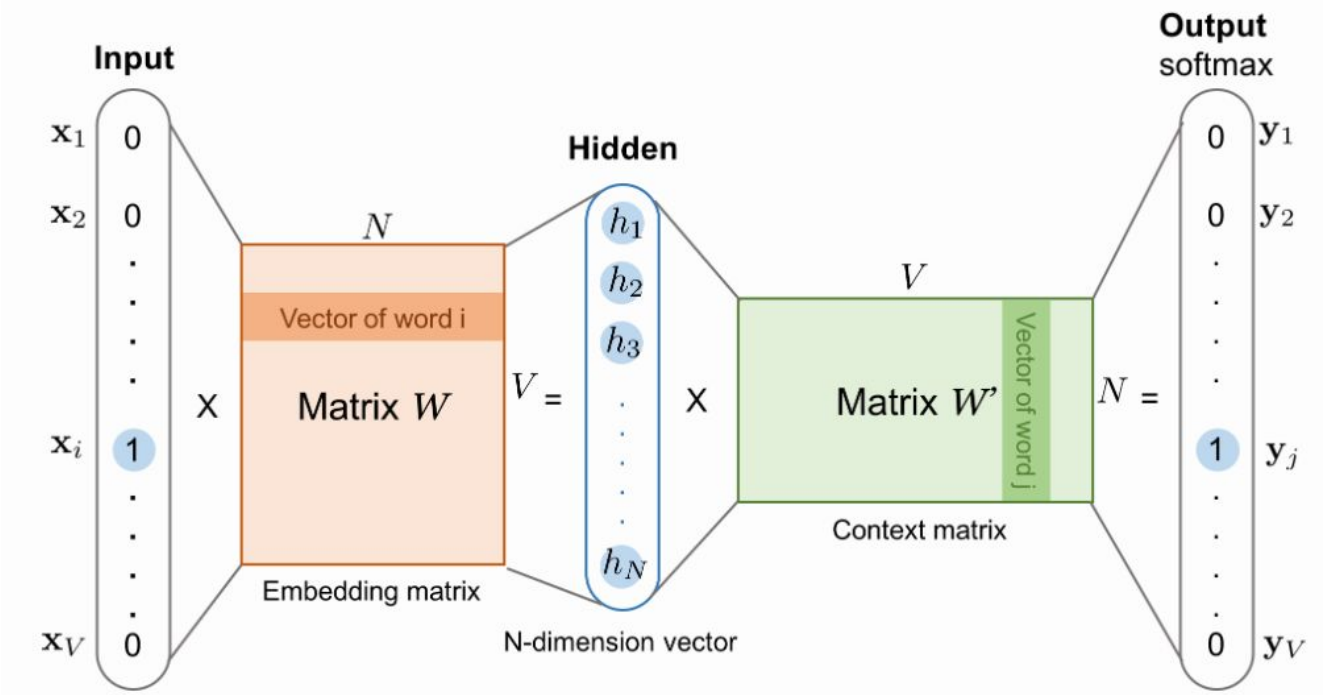
Текст в вектора (one-hot encoding)

Feature (Color)
Red
Green
Yellow
Green
Red

One Hot Encoding →

One Hot Encoded Vector	Red	Green	Yellow
[1,00]	1	0	0
[0,1,0]	0	1	0
[0,0,1]	0	0	1
[0,1,0]	0	1	0
[1,00]	1	0	0

Word2Vec



Матрица W (NxV)
N - размер вектора
V - длина словаря

вектор i - строка матрицы W
вектор j - колонка матрицы W'

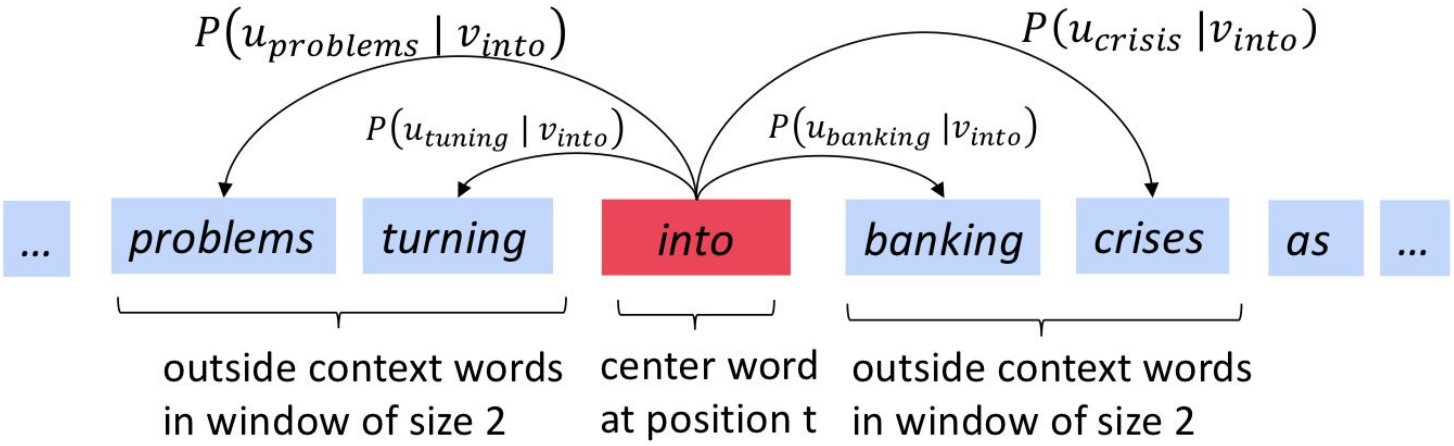
на выходе получаем скалярное произведение векторов softmax(i*j)

Задача чтобы скалярное произведение было как можно больше

loss - Cross Entropy

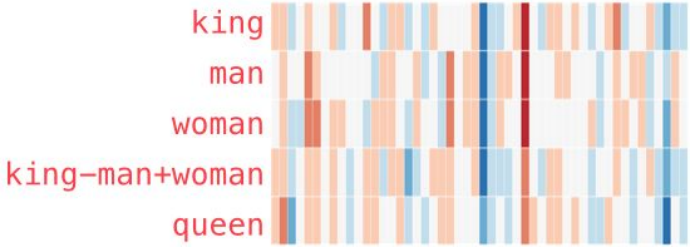
https://mohitmayank.com/a_lazy_data_science_guide/natural_language_processing/word2vec/

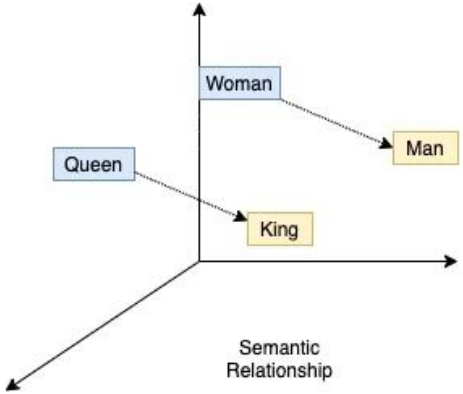
Word2Vec

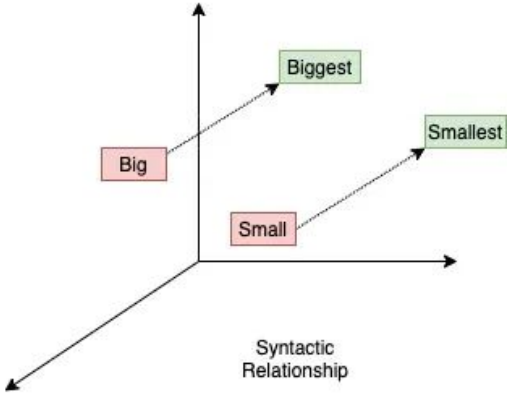

$P(u_{problems} \mid v_{into})$
$P(u_{crisis} \mid v_{into})$
$P(u_{tuning} \mid v_{into})$
$P(u_{banking} \mid v_{into})$
...
problems
turning
into
banking
crises
as
...
outside context words in window of size 2
center word at position t
outside context words in window of size 2

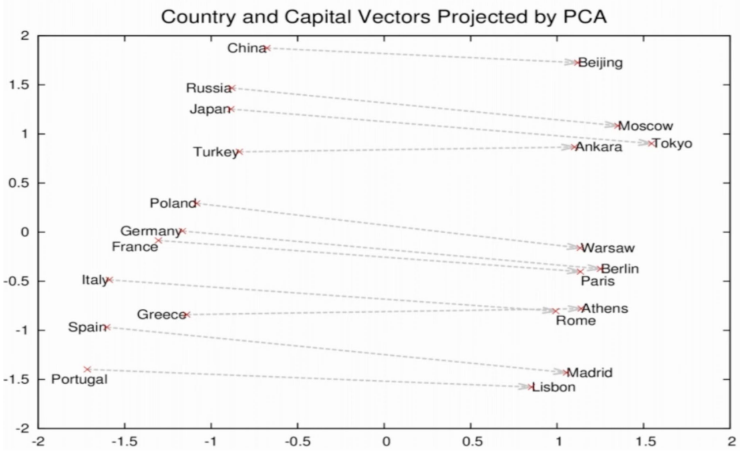
Word2Vec

king - man + woman ~= queen









Word2Vec

🔁 **Аналогии**:
`"king" - "man" + "woman" ≈ "queen"`

🔍 **Поиск похожих слов**:
`"happy"` → `"joyful"`, `"glad"`

🧠 **Кластеризация тем**:
Группировка слов по смыслу (спорт, еда, политика)

🗣️ **Понимание текста**:
Средний вектор предложения → классификация тональности
`"Сервис был отвратительный, официант грубый."` → `"негатив"`

🤖 **Вход для нейросетей**:
Эмбеддинги вместо one-hot для RNN, CNN, трансформеров

Word2Vec

Плюсы:

- Векторы отражают смысл слов
- Размерность вектора не зависит от размера словаря
- При добавлении документов векторы можно дообучать

Недостатки:

- фиксированный набор словаря
- для редких слов эмбеддинги получаются не оптимальные
- нужны предобработки т.к слова с разными окончаниями будут разными (Лемматизация, стемминг)

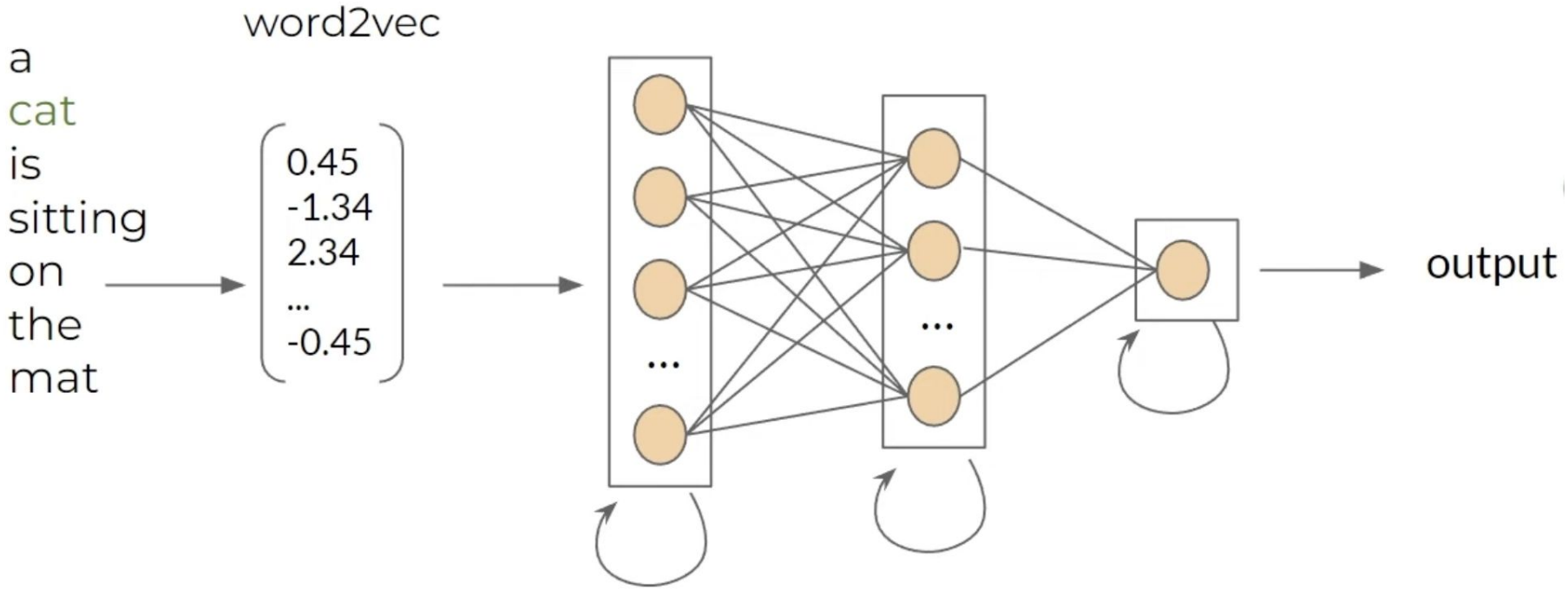
Стемминг – это грубый эвристический процесс, который отрезает «лишнее» от корня слов, часто это приводит к потере словообразовательных суффиксов.

Лемматизация – это более тонкий процесс, который использует словарь и морфологический анализ, чтобы в итоге привести слово к его канонической форме – лемме.

Материалы

https://jramkiss.github.io/2019/08/21/word-embeddings/
https://arxiv.org/pdf/1301.3781.pdf
https://habr.com/ru/articles/446530/
https://towardsdatascience.com/word2vec-research-paper-explained-205cb7eecc30
https://www.youtube.com/watch?v=WbtQzAvhnRI
https://mohitmayank.com/a_lazy_data_science_guide/natural_language_processing/word2vec/

Recurrent neural network (RNN)

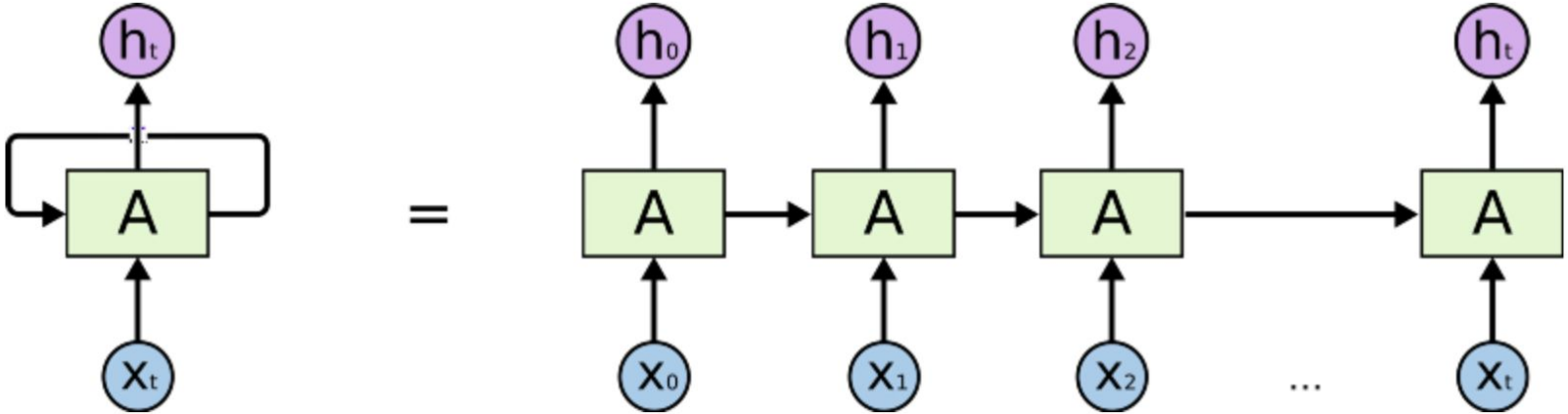


Рекуррентный слой принимает текущее состояние + вектор состояния с предыдущего шага

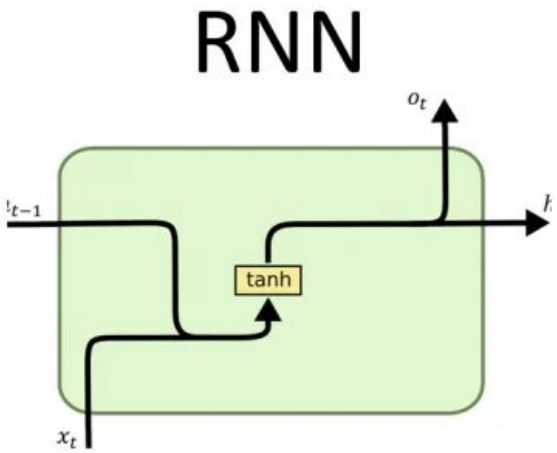
Recurrent neural network (RNN)

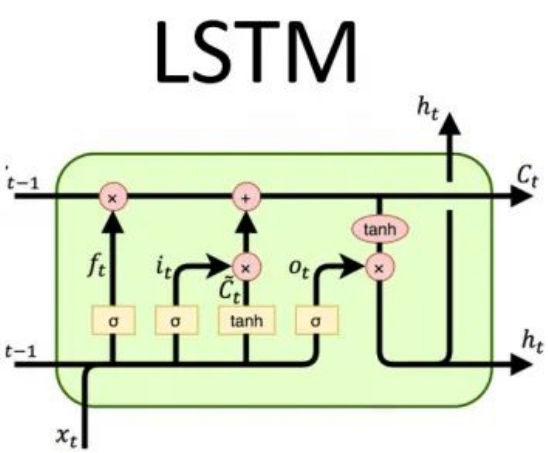
Рекуррентный слой

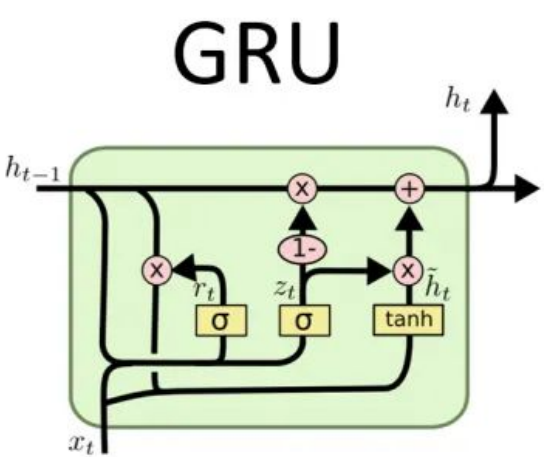
Рекуррентный слой развернутый во времени



Вариации слоев







http://vbystricky.ru/2021/05/rnn_lstm_gru_etc.html
https://youtu.be/3OljkWQ2Uc0?si=K_JzNqXz0ToSFSfb

Типы рекуррентных сетей

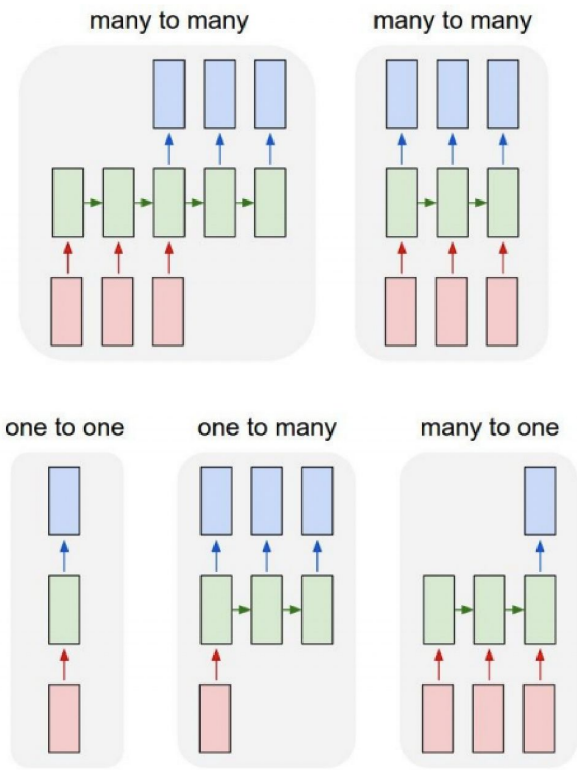
Many to Many – используется, например, в задачах **машинного перевода**, где на вход подается последовательность слов на одном языке, а на выходе — последовательность слов на другом языке.

Many to One – применяется в задачах **анализа тональности текста**, где весь текст (последовательность слов) используется для предсказания одной категории: положительный, нейтральный или отрицательный.

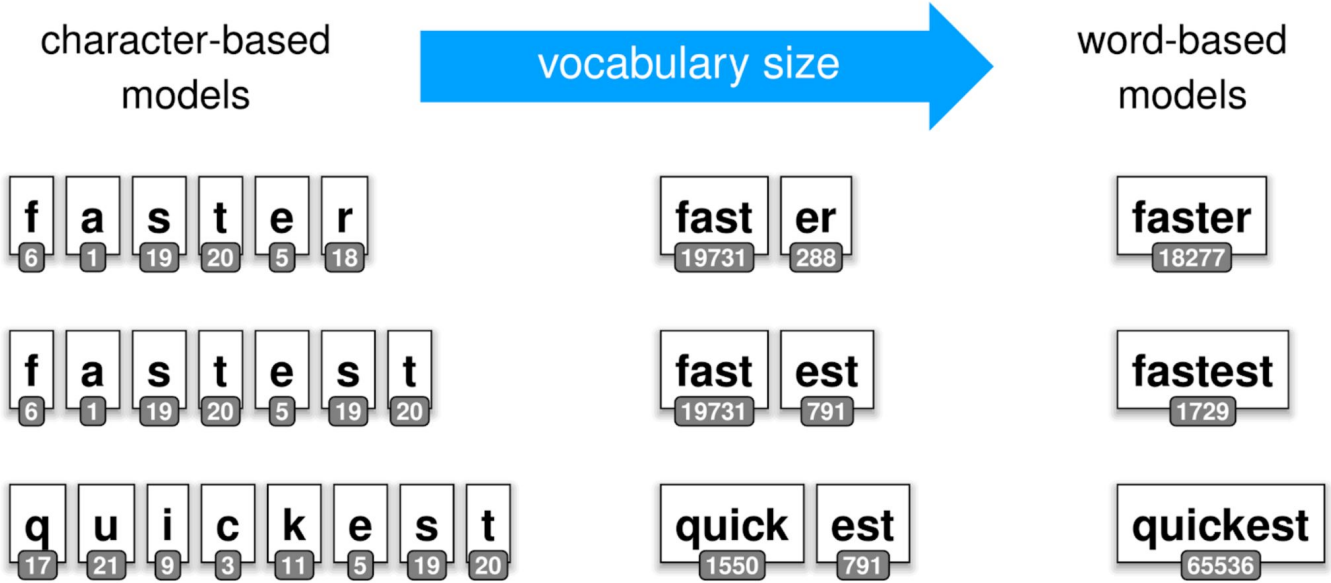
One to Many – подходит для **генерации текста** по изображению или **автодополнения**, где на вход подаётся один вектор (например, закодированное изображение), а на выходе — последовательность слов.

One to One – относительно редкая архитектура, может использоваться для **нелинейных рекуррентных вычислений**, когда каждый вход связан с одним выходом, например, в некоторых вариантах обработки временных рядов.

Основным недостатком простых рекуррентных сетей является забывание начала предложения



Tokenization



https://platform.openai.com/tokenizer tokenizer gpt
https://huggingface.co/docs/transformers/tokenizer_summary tokenizer_summary
https://youtu.be/iOrNbK2T92M?si=j7L2c_EiCqtxEuho Igor Kotenkov
https://youtu.be/zduSFxRajkE?si=i-63AmBDb1aZ-mhH Andrej Karpathy

Attention is all you need

1. Feed Forward
2. Multi-Head Attention
3. Masked Multi-Head Attention
4. Add & Norm
5. Positional Encoding

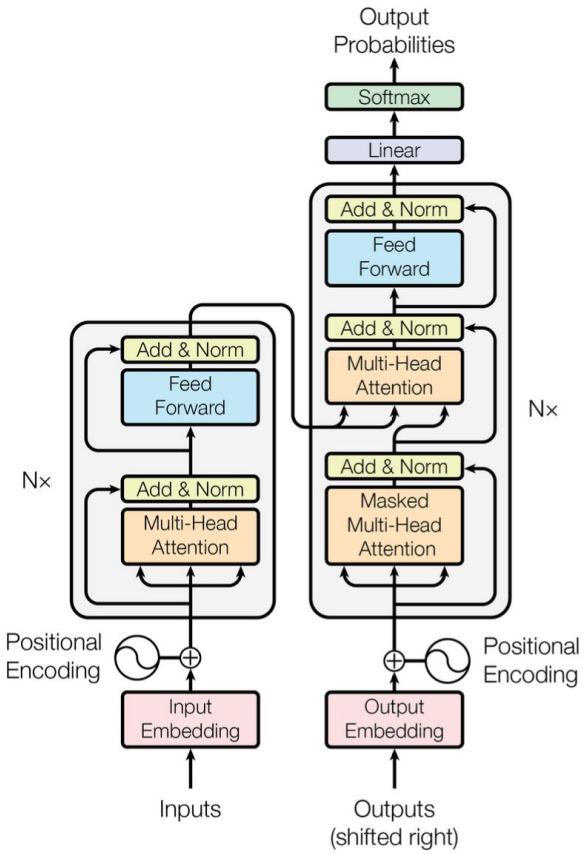
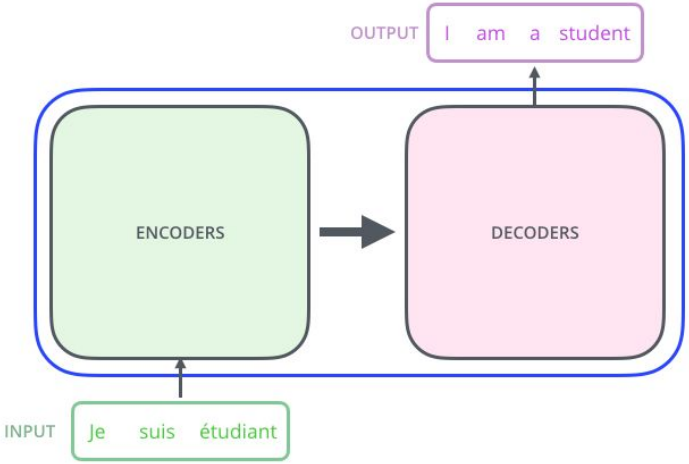


Figure 1: The Transformer - model architecture.

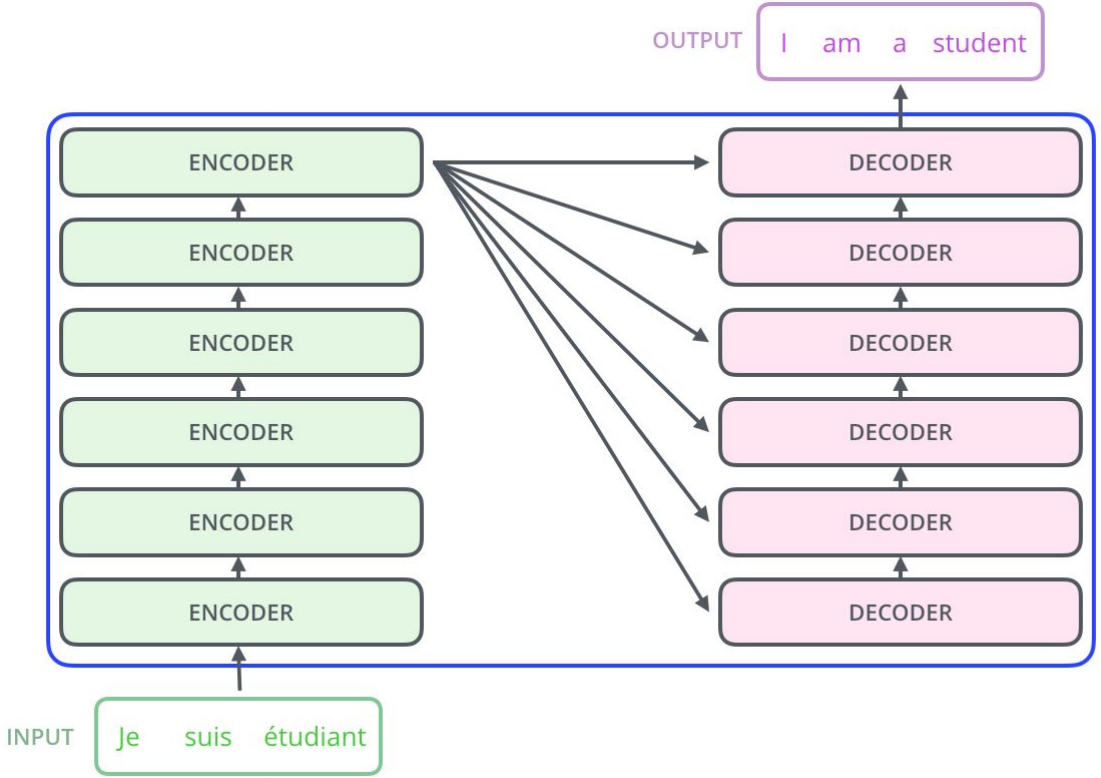
https://jalammar.github.io/illustrated-transformer/
https://arxiv.org/abs/1706.03762

Transformer

Архитектура взялась из задачи перевода текста



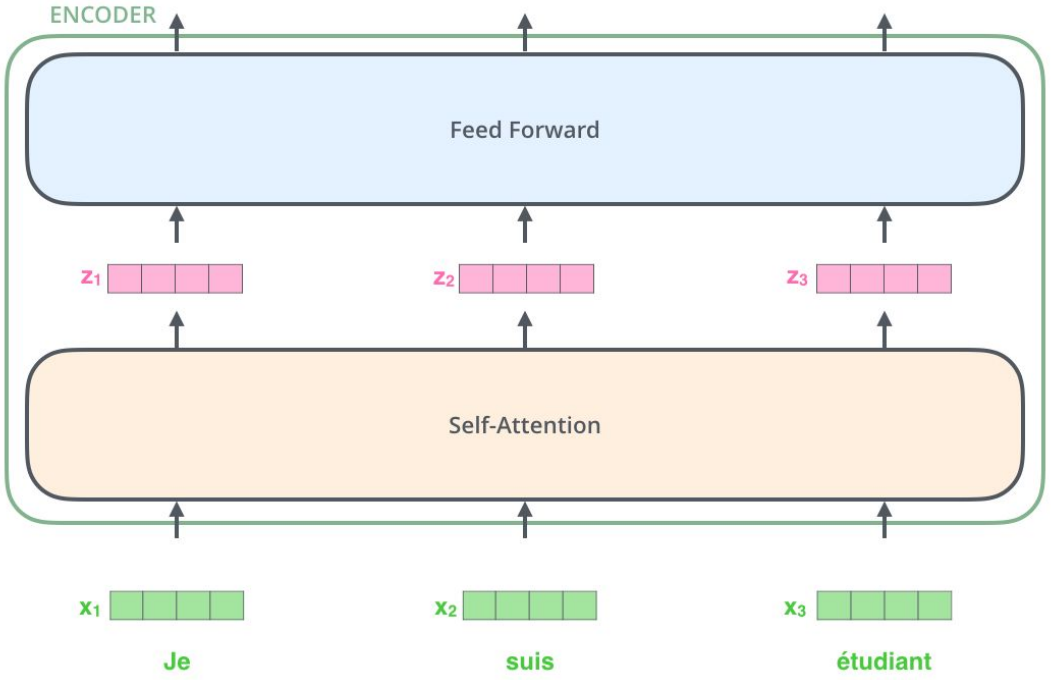
Transformer



- На вход подается тензор размера [batch, length, Edim]
- Размерность выхода каждого каждого энкодера должна соответствовать входу
- Выход и энкодеров далее подается в декодеры

Encoder block

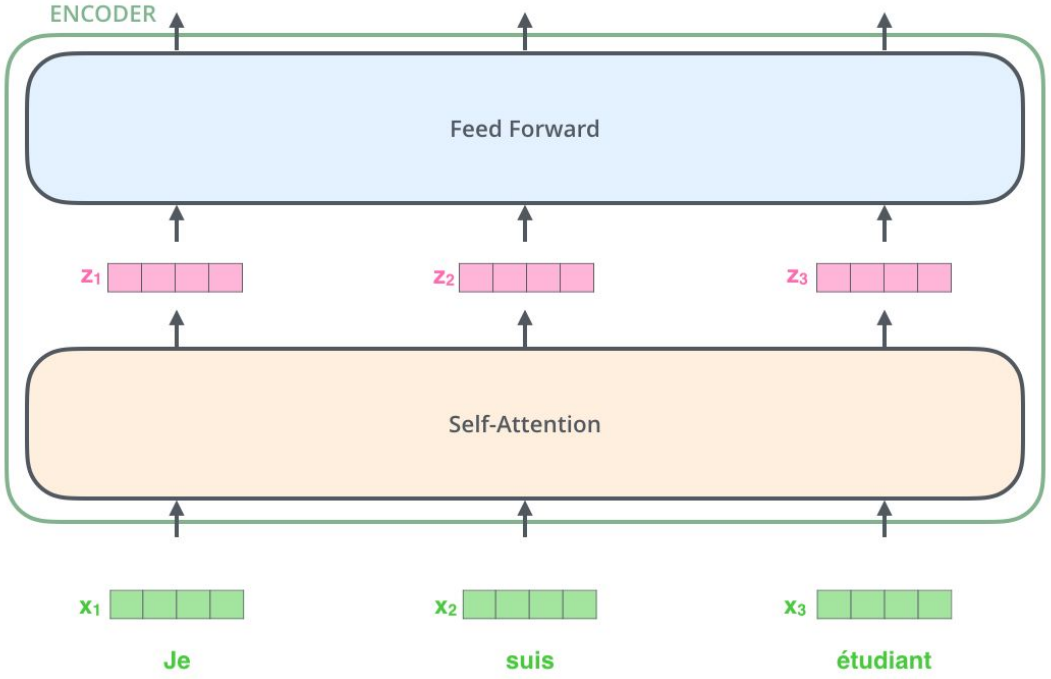
В каждом блоке – слой self-attention и feedforward layer, который независимо применяется к каждой позиции входа



Encoder block

В каждом блоке – слой self-attention и feedforward layer, который независимо применяется к каждой позиции входа

Computation

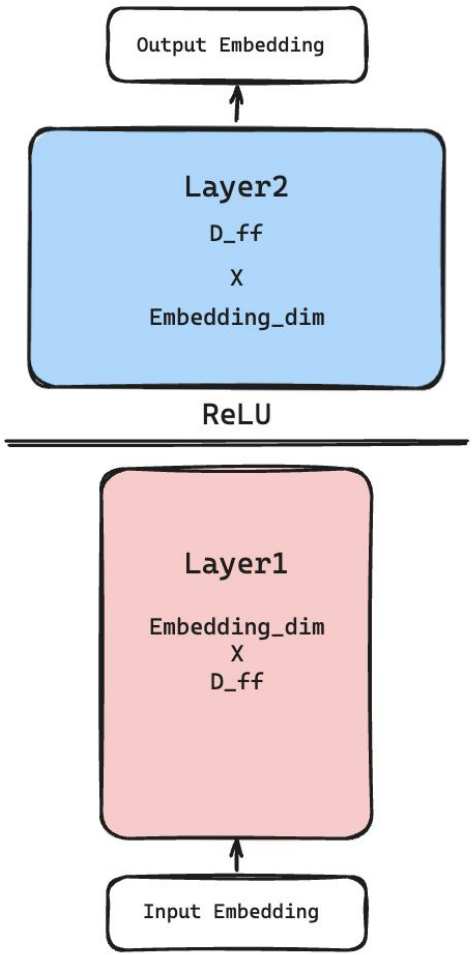


Communication

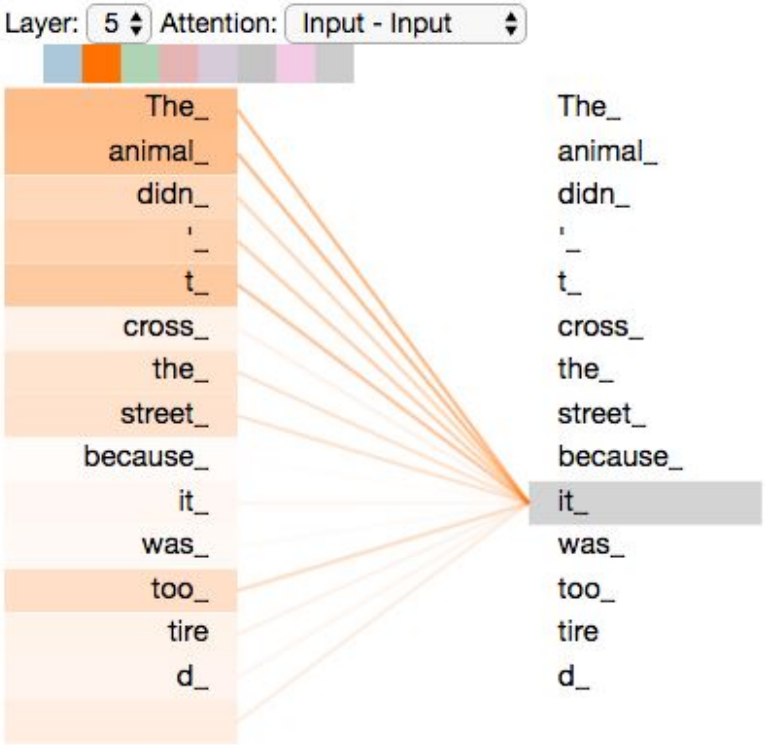
Feed forward layer

Feed-forward layer eats 2/3 of the model params!
Embedding_dim -> D_ff -> Embedding_dim
D_ff (x4) от Embedding_dim

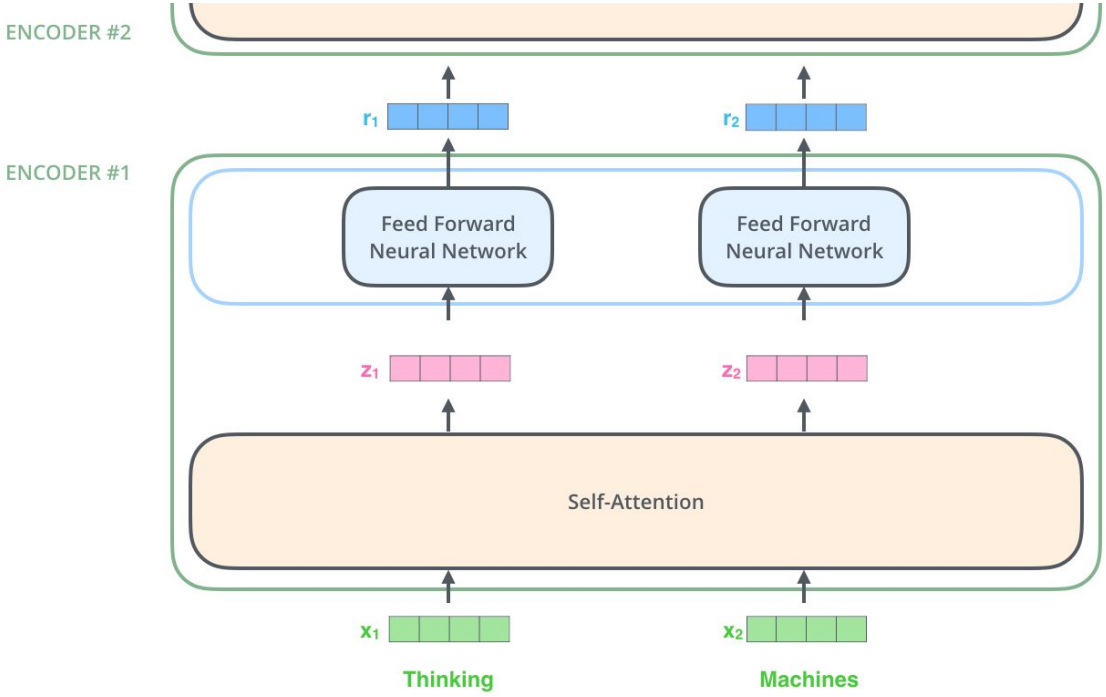
Typical values BERT-base
Embedding_dim= 768
D_ff = 3072



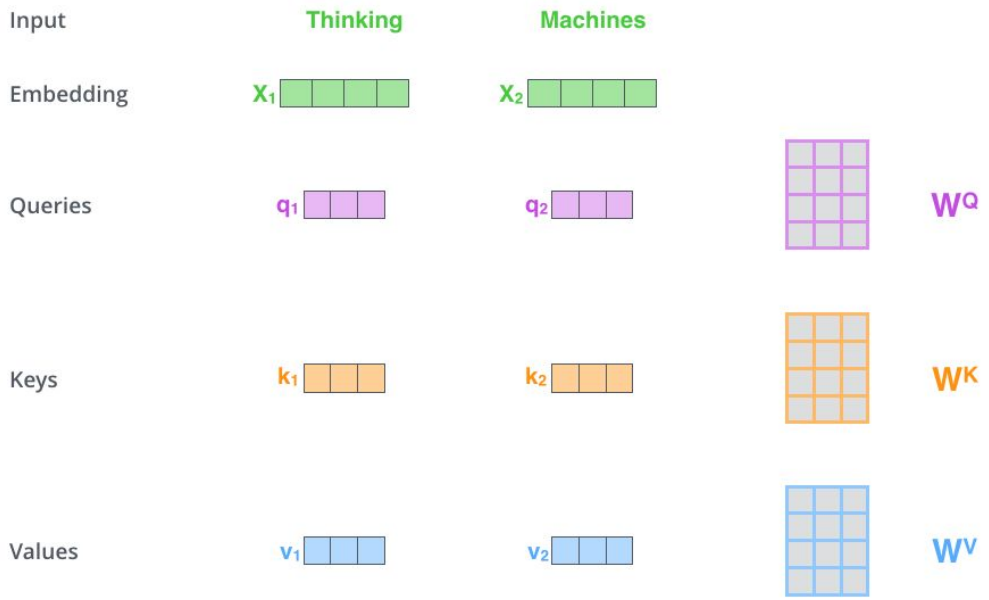
Self-Attention



Self-Attention



Self-Attention



$$x - (1, n)$$

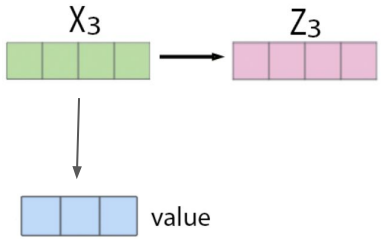
$$Wq\,,\ Wq\,,\ Wq\,, \ - (n, m)$$

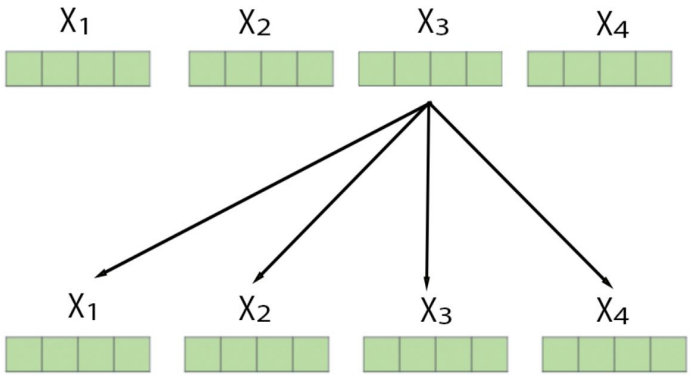
$$q = x * Wq$$

$$k = x * Wk$$

$$v = x * Wv$$

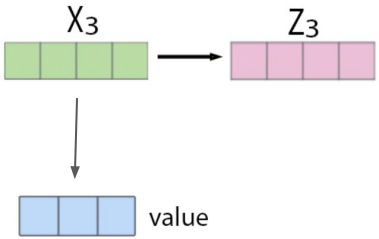
Self-Attention

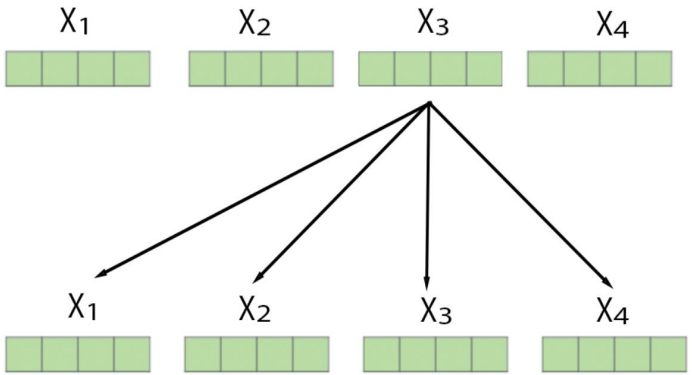




$$Z_3 = \sum_{i=1}^{4} a_i * v_i \quad ; \quad \sum_{i=1}^{4} a_i = 1$$

Self-Attention



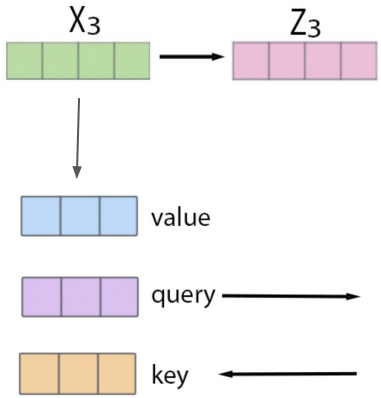


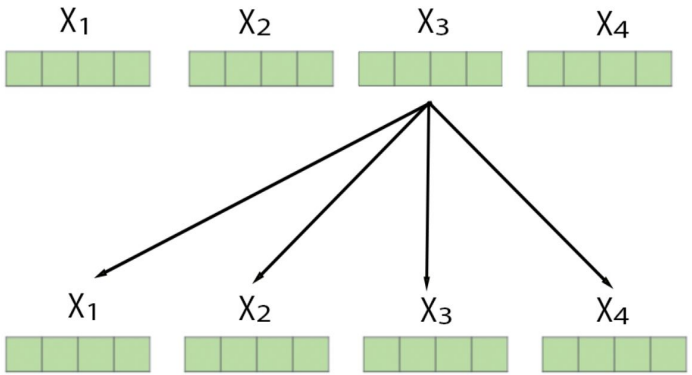
$$Z_3 = \sum_{i=1}^{4} a_i * v_i \quad ; \quad \sum_{i=1}^{4} a_i = 1$$

$$\left\{\widehat{a_i}\right\}_{i=1}^{4} \xrightarrow{\text{Softmax}} \left\{a_i\right\}_{i=1}^{4}$$

$$\widehat{a_i} = f(x_3, x_i)$$

Self-Attention



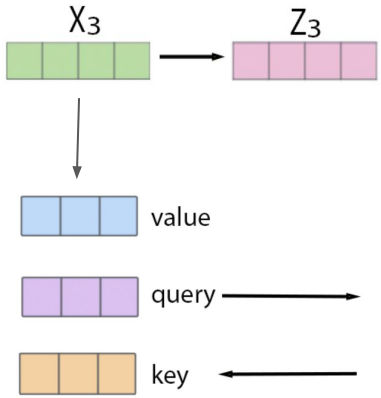


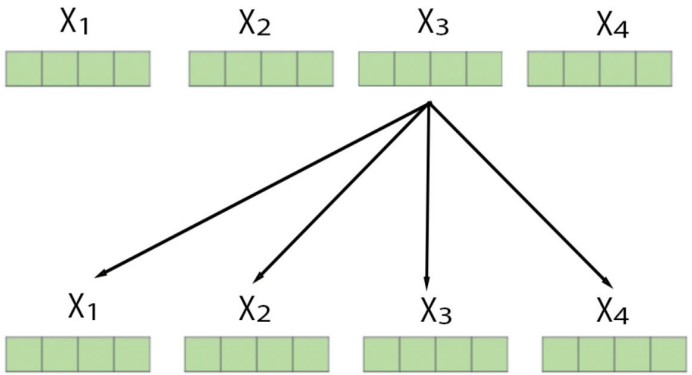
$$Z_3 = \sum_{i=1}^{4} a_i * v_i \quad ; \quad \sum_{i=1}^{4} a_i = 1$$

$$\left\{\widehat{a_i}\right\}_{i=1}^{4} \xrightarrow{\text{Softmax}} \left\{a_i\right\}_{i=1}^{4}$$

$$\widehat{a_i} = f(x_3, x_i)$$

Self-Attention



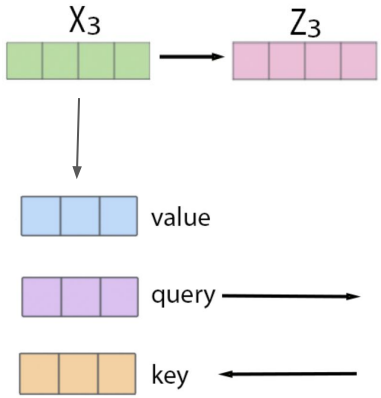


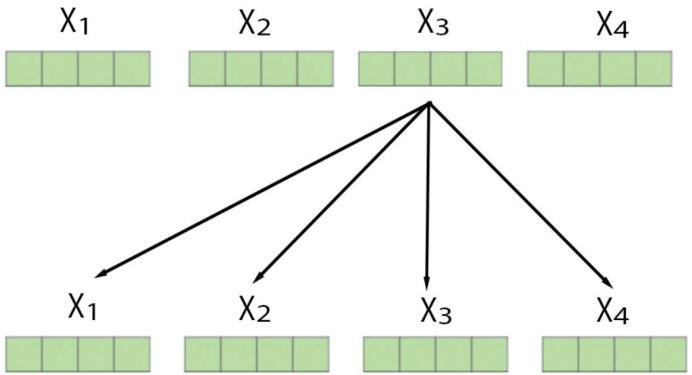
$$Z_3 = \sum_{i=1}^{4} a_i * v_i \quad ; \quad \sum_{i=1}^{4} a_i = 1$$

$$\left\{\widehat{a_i}\right\}_{i=1}^{4} \xrightarrow{\text{Softmax}} \left\{a_i\right\}_{i=1}^{4}$$

$$\widehat{a_i} = f(x_3, x_i) \qquad \widehat{a_i} = \langle q_3, k_i \rangle$$

Self-Attention





$$Z_3 = \sum_{i=1}^{4} a_i * v_i \quad ; \quad \sum_{i=1}^{4} a_i = 1$$

$$\left\{\widehat{a_i}\right\}_{i=1}^{4} \xrightarrow{\text{Softmax}} \left\{a_i\right\}_{i=1}^{4}$$

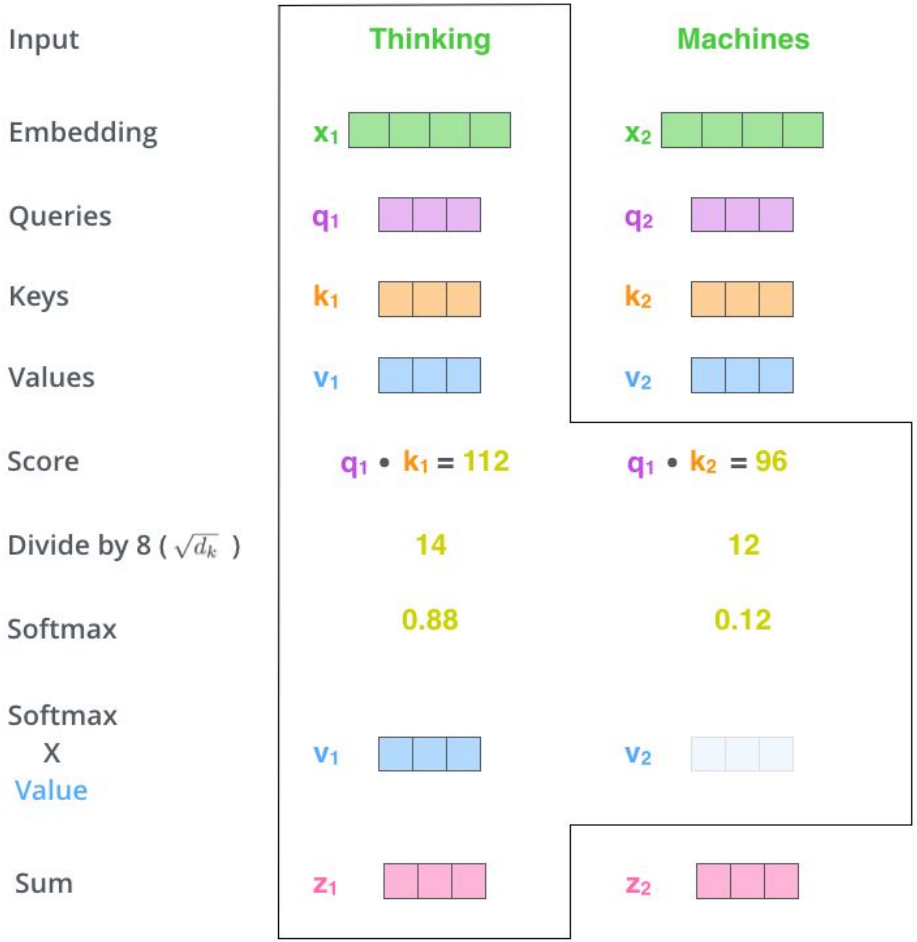
$$\widehat{a_i} = f(x_3, x_i) \qquad \widehat{a_i} = \ < q_3, k_i >$$

$$\left\{\widehat{a_i}\right\}_{i=1}^{4} = \left\{< q_3, k_1 >, \ < q_3, k_2 >, \ < q_3, k_3 >, \ < q_3, k_4 >\right\}$$

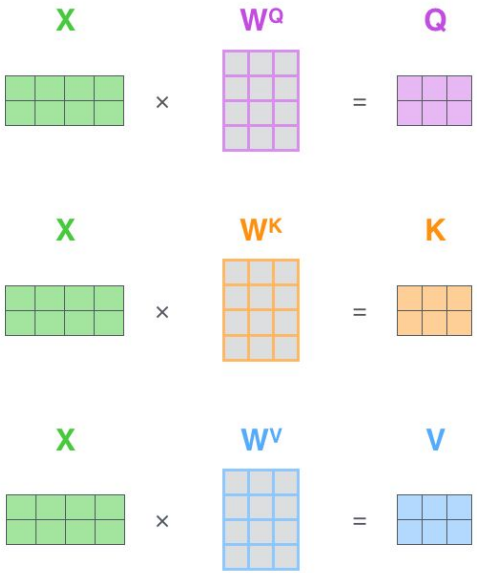
$$Z_1 = a_1 * v_1 + a_2 * v_2$$

$$Z_1 = < q_1 * k_1 > * v_1 + < q_1 * k_2 > * v_2$$

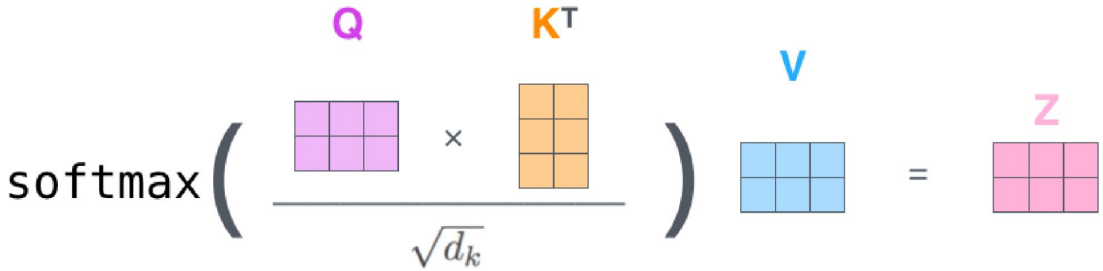
Делим на квадратный корень из размерности чтобы не увеличивать дисперсию



Self-Attention

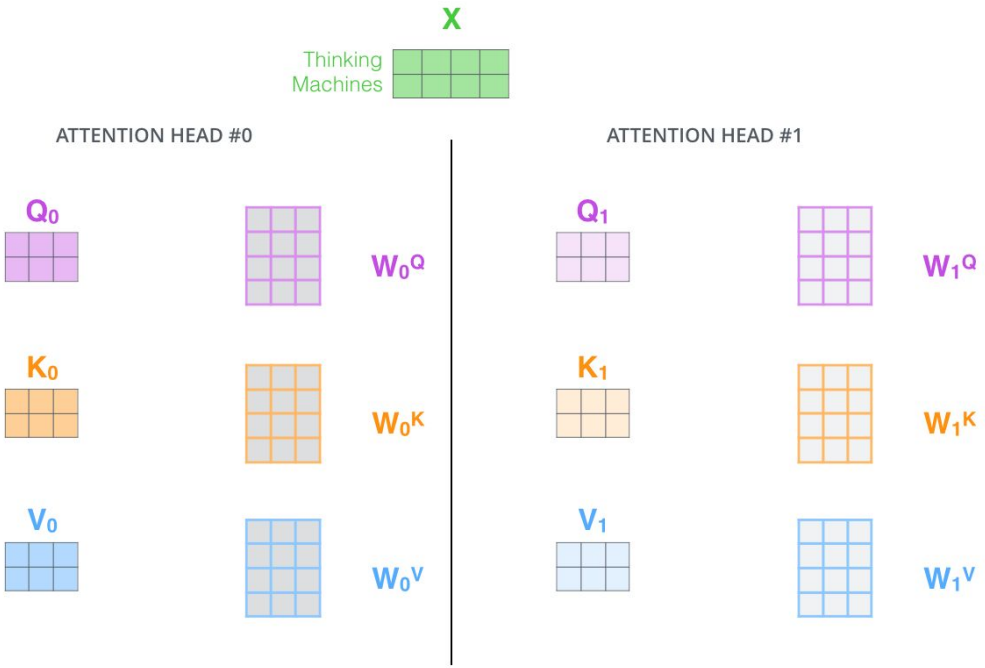


Финальная формула

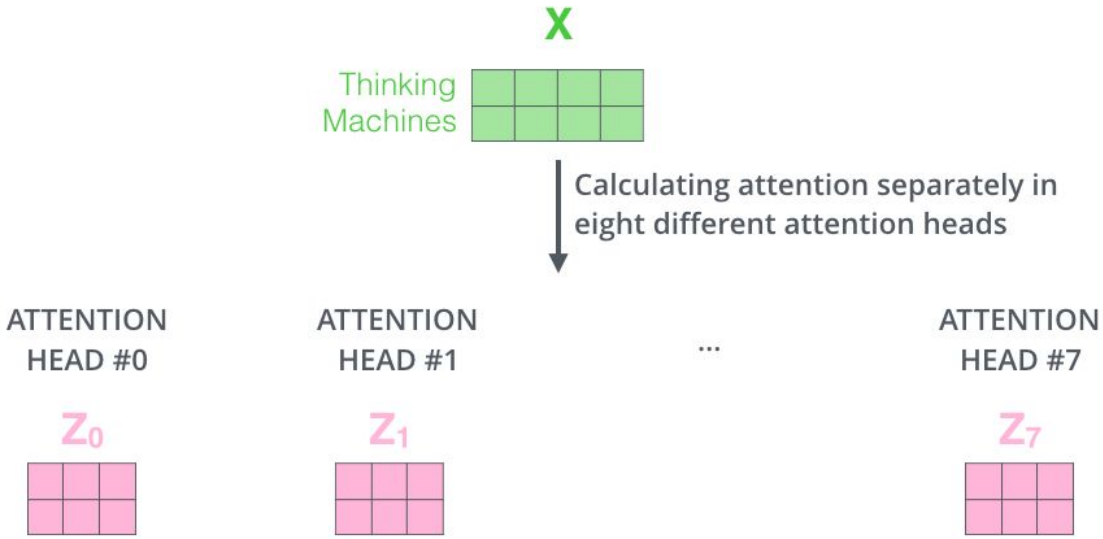


$$Z_3 = \sum_{i=1}^{4} a_i * v_i$$

Multi-Head Attention



Multi-Head Attention

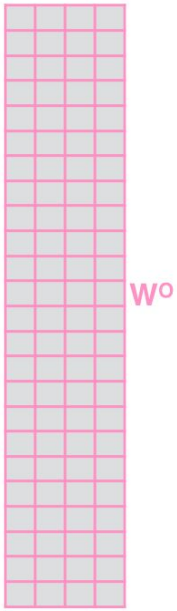


Multi-Head Attention

1) Concatenate all the attention heads

2) Multiply with a weight matrix W^O that was trained jointly with the model

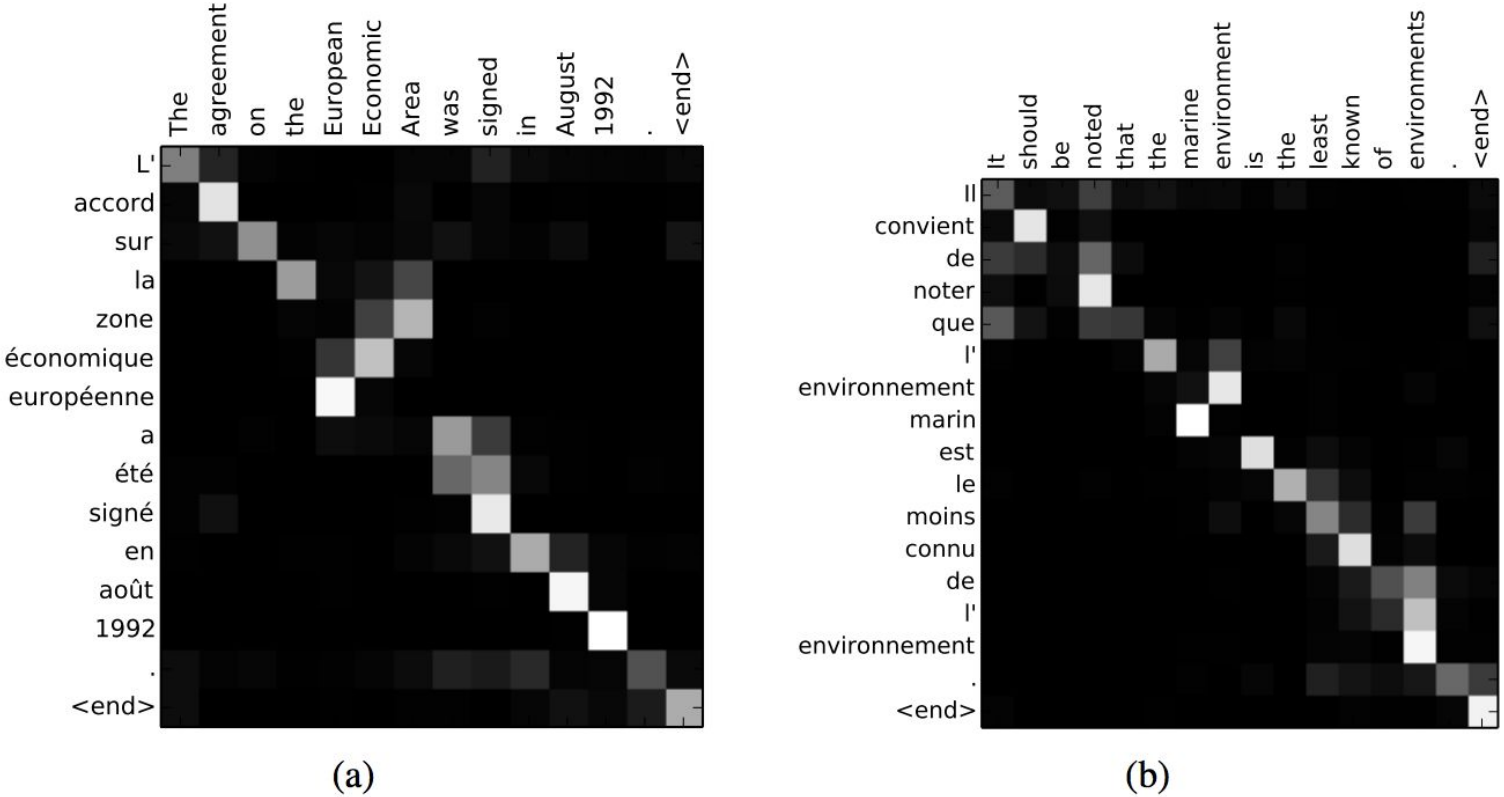
X



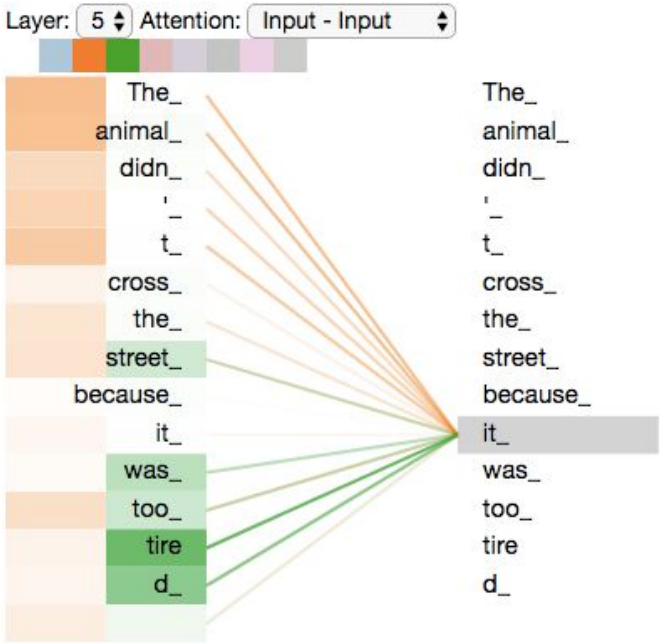
3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN

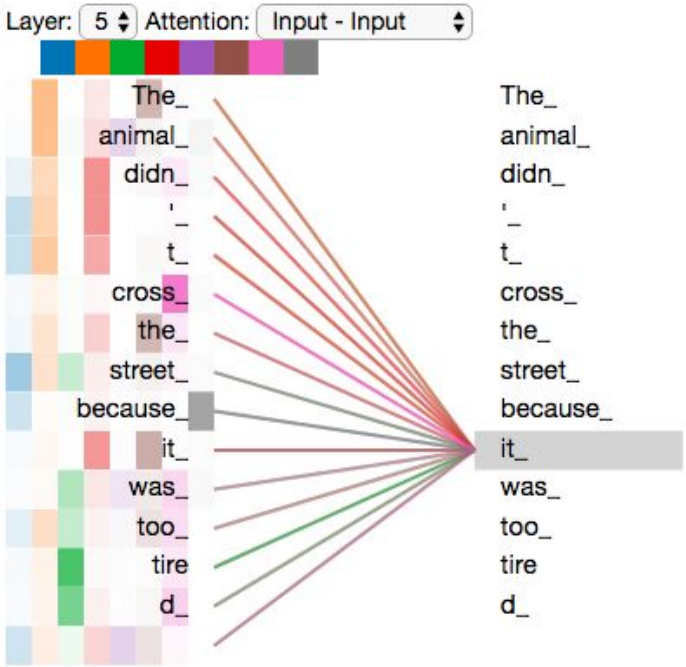
=

Attention visualization



Multi-Head Attention





Визуализация разных голов

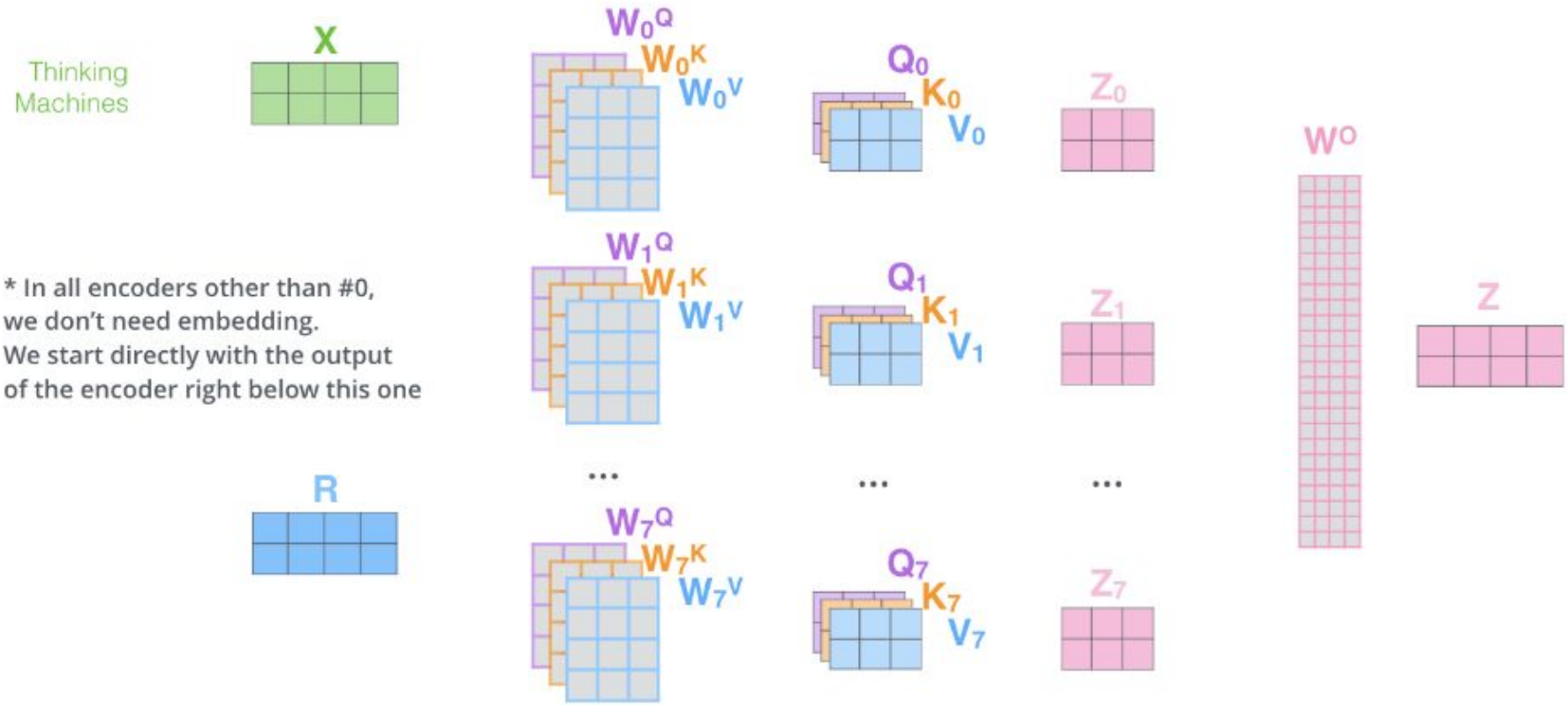
1) This is our input sentence*

2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

4) Calculate attention using the resulting Q/K/V matrices

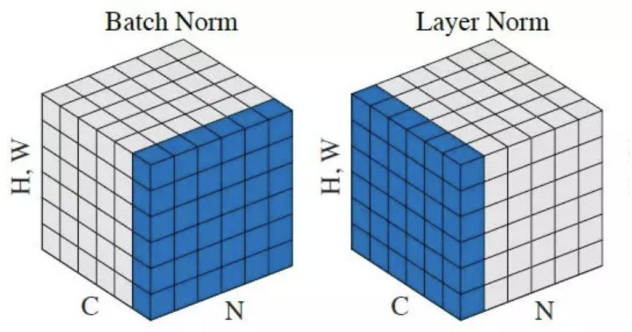
5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

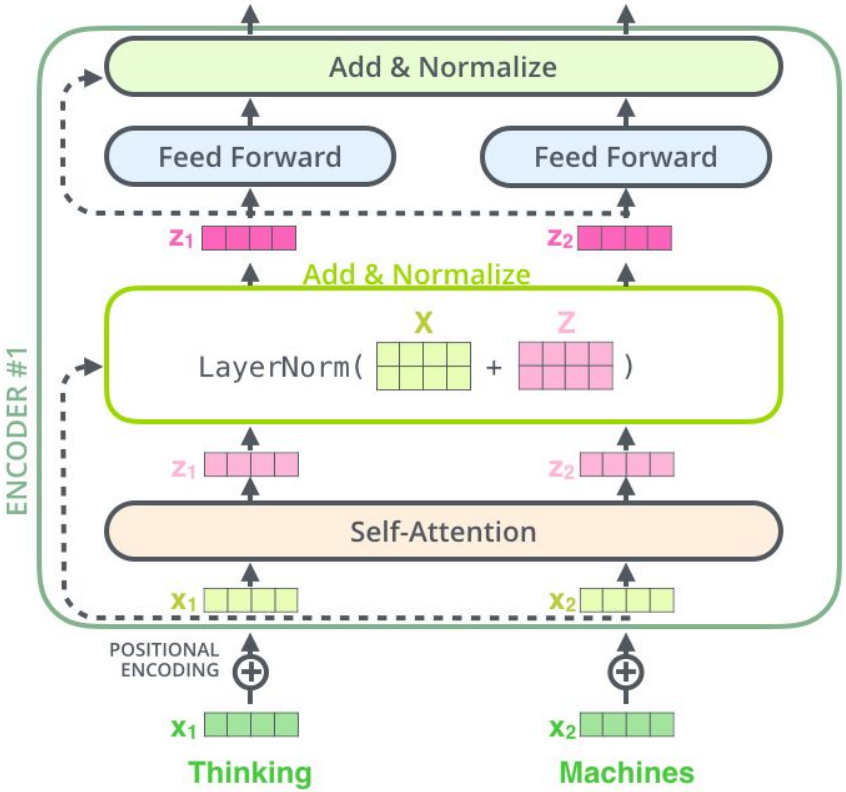


* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

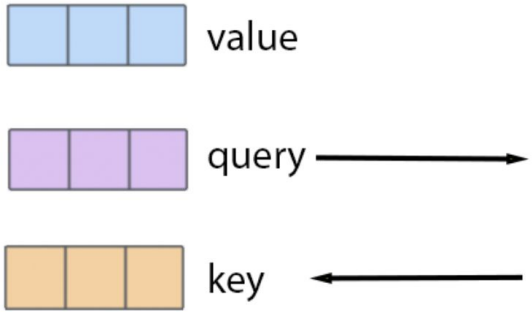
Add & Norm

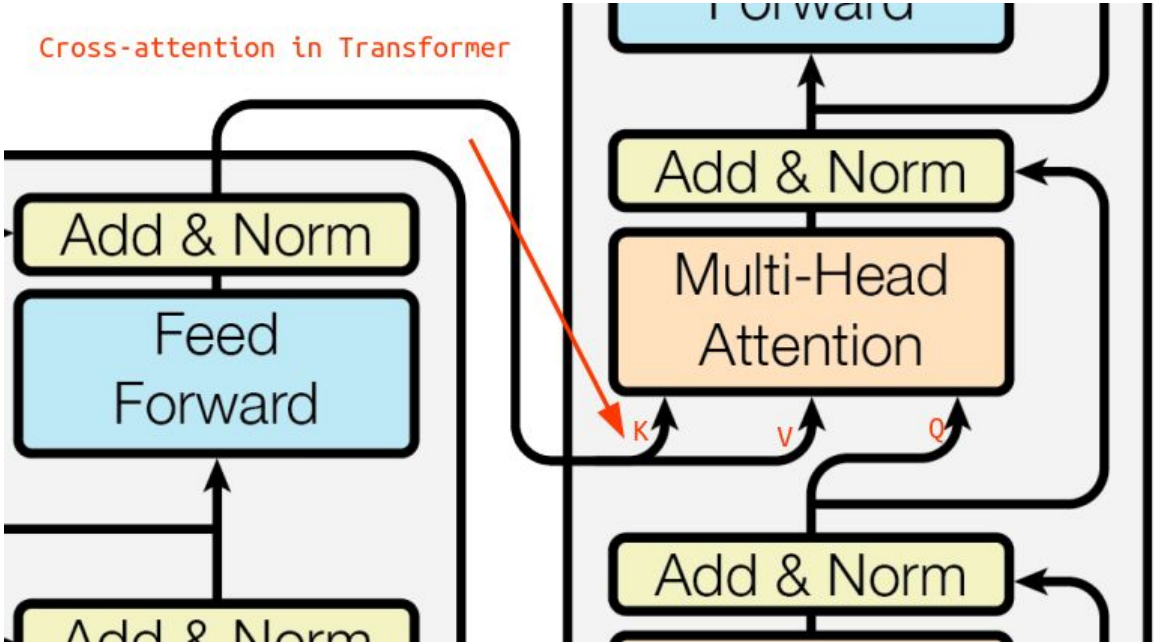
1. Residual connections
2. Normalize to N(0,1) к нулевой мат ожиданием и дисперсией
3. В Layer norm нормализация по фичам



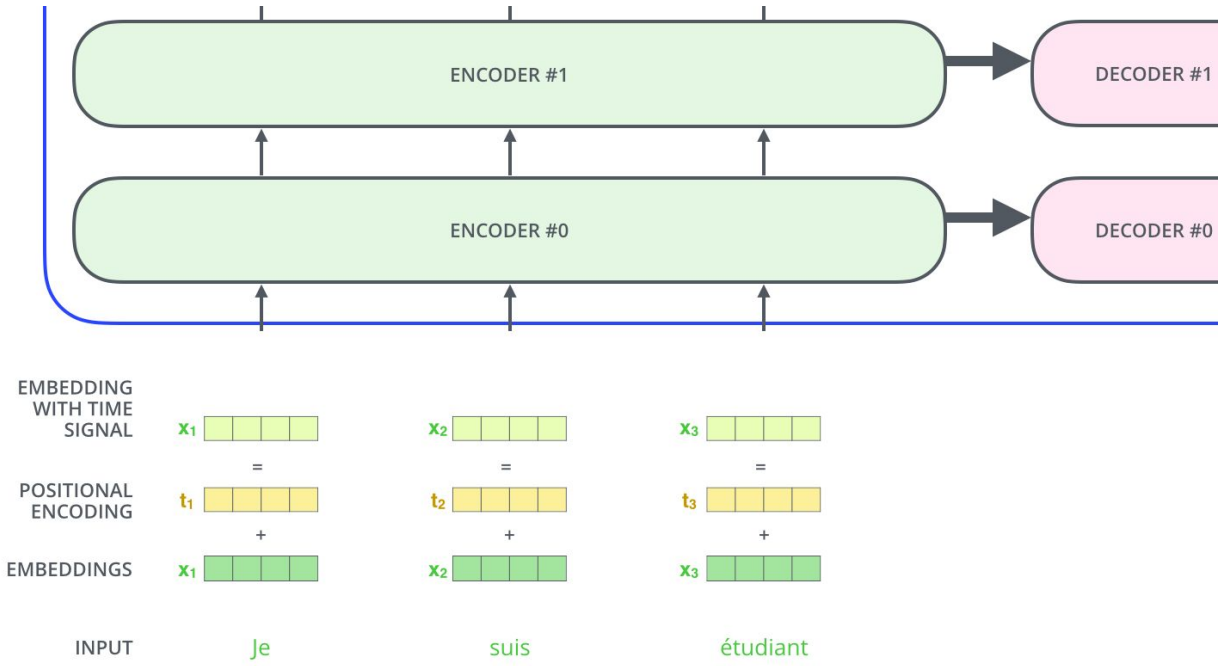


Cross Attention



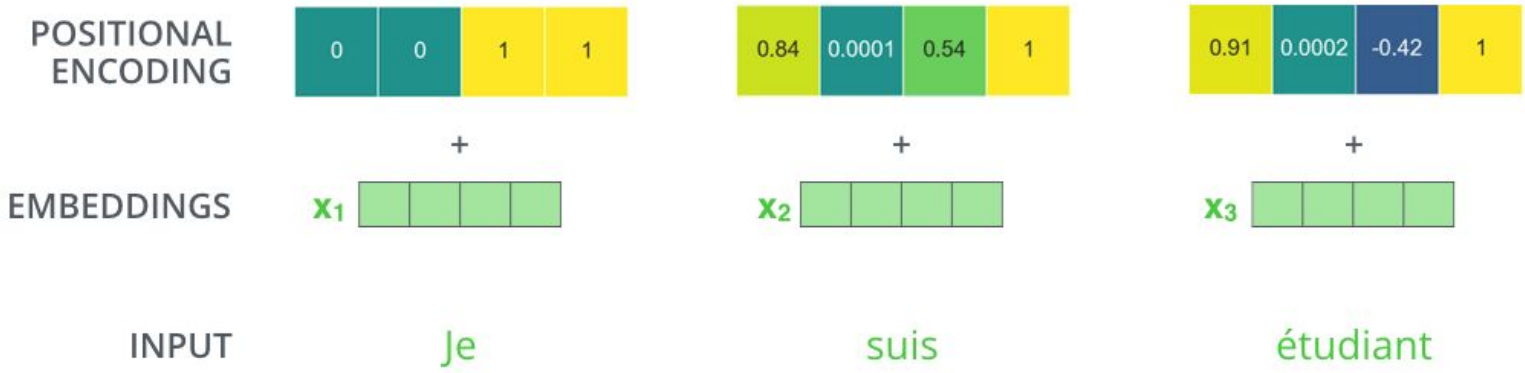


Positional encoding



Positional encoding

Если в качестве примера размер эмбединга будет равен 4 то то positional вектор будет выглядеть следующим образом



Positional encoding

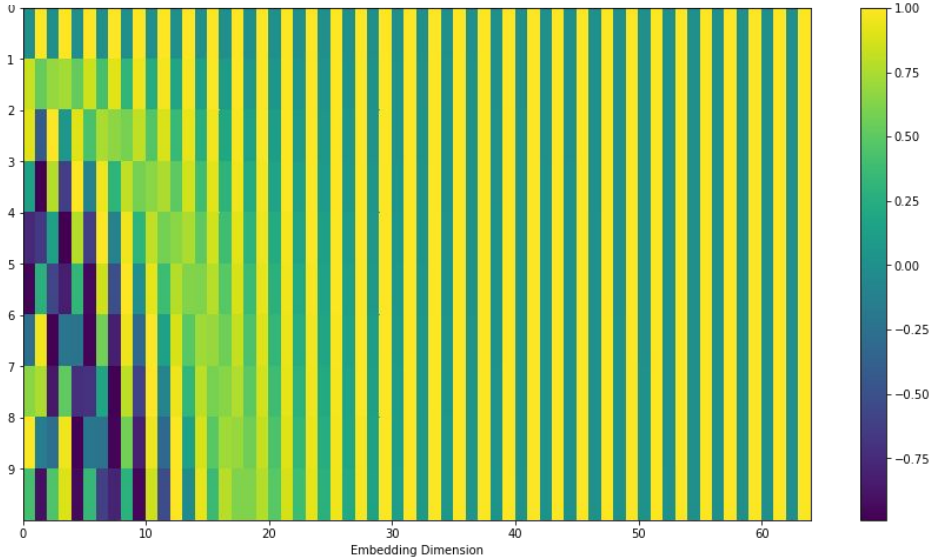
t - номер слова

ω_k частота

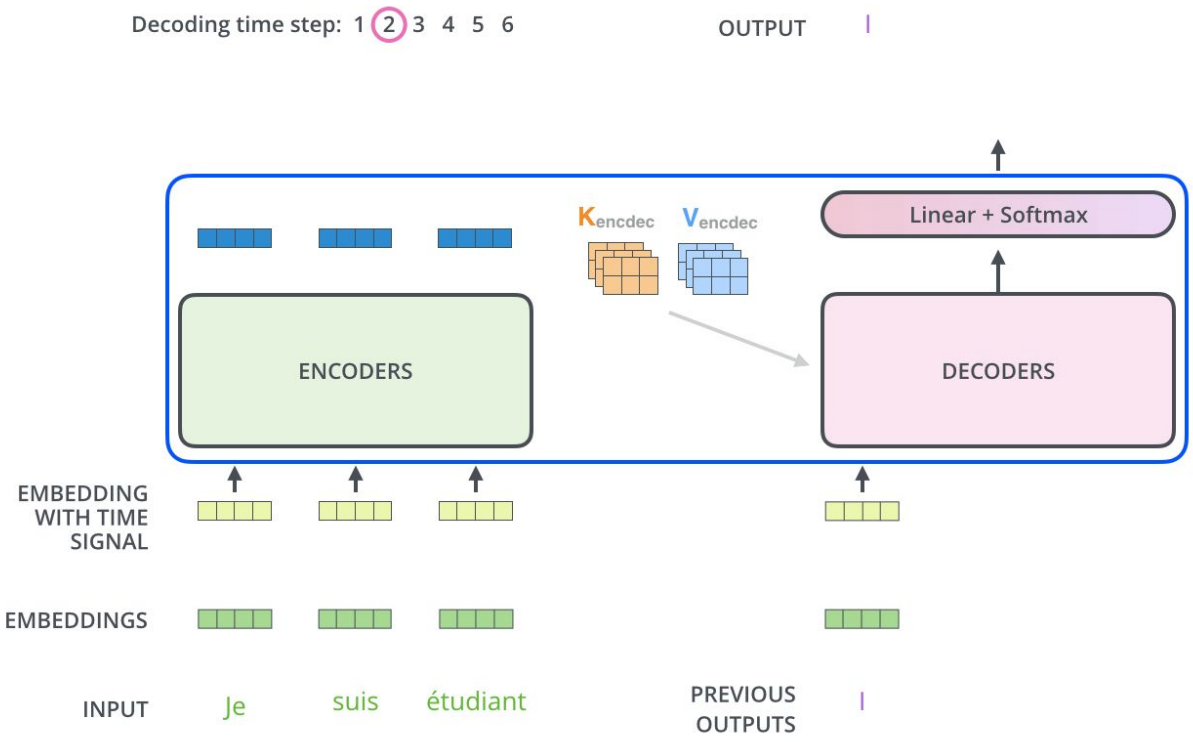
Главный плюс этого кодирования что норма этого вектора всегда постоянная независимо от t

$$\overrightarrow{p_t}^{(i)} = f(t)^{(i)} := \begin{cases} \sin(\omega_k . t), & \text{if } i = 2k \\ \cos(\omega_k . t), & \text{if } i = 2k+1 \end{cases} \quad \omega_k = \frac{1}{10000^{2k/d}}$$

$$\overrightarrow{p_t} = \begin{bmatrix} \sin(\omega_1 . t) \\ \cos(\omega_1 . t) \\ \\ \sin(\omega_2 . t) \\ \cos(\omega_2 . t) \\ \\ \vdots \\ \\ \sin(\omega_{d/2} . t) \\ \cos(\omega_{d/2} . t) \end{bmatrix}_{d \times 1}$$



Masked Multi-Head Attention



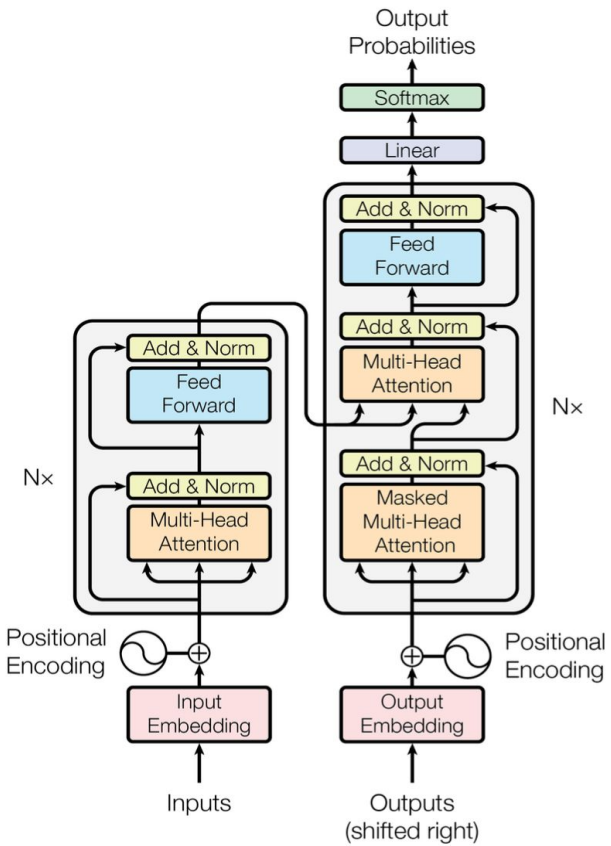
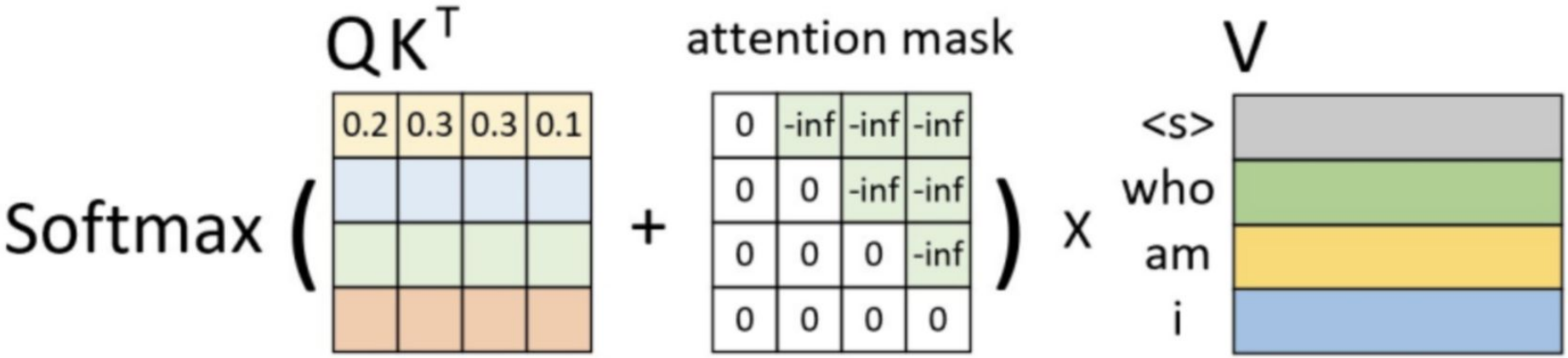


Figure 1: The Transformer - model architecture.

https://github.com/hyunwoongko/transformer Code

Masked Multi-Head Attention



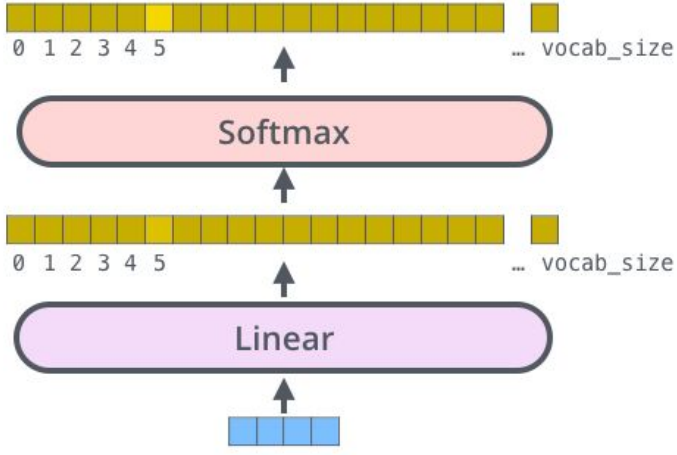
Loss function

Which word in our vocabulary is associated with this index?

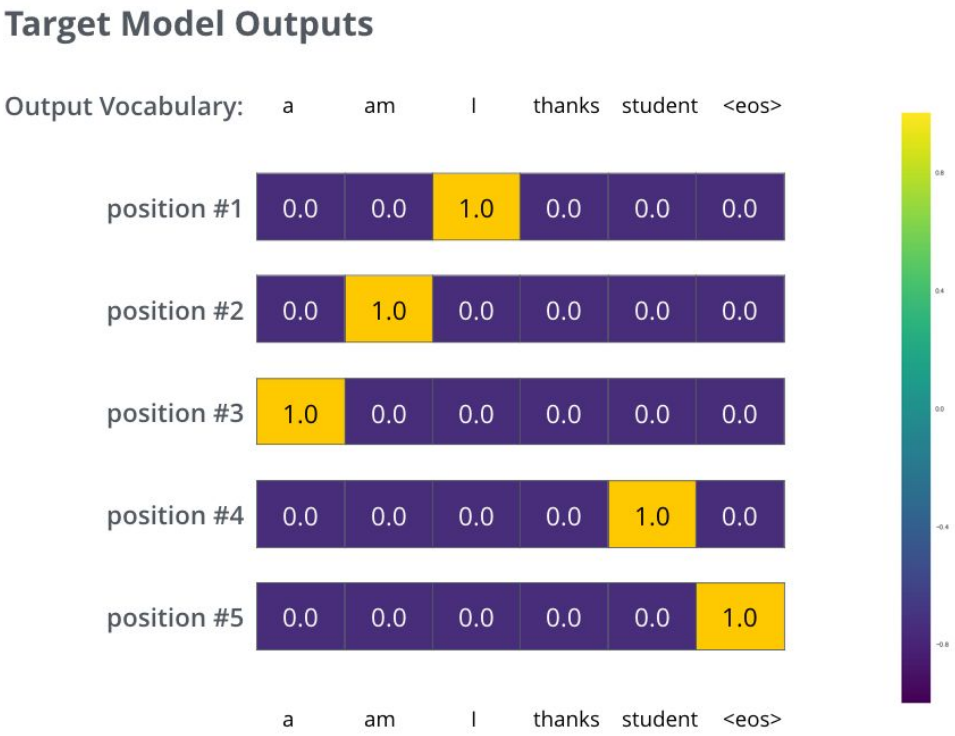
am

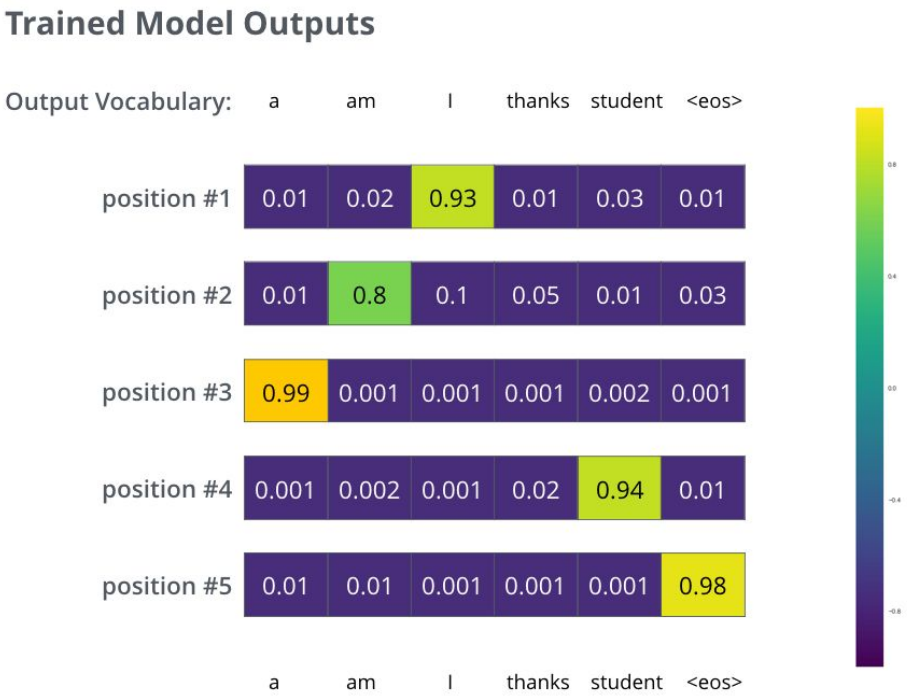
Get the index of the cell with the highest value (argmax)

5



Loss function



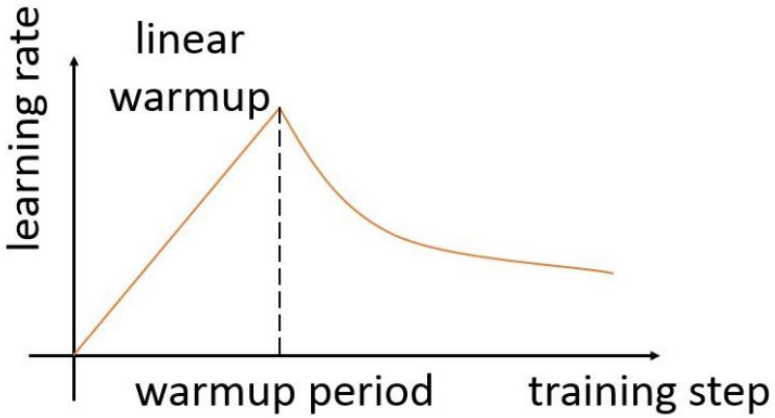


Training

warmup time

Одна из особенностей это увеличенный learning rate в начале обучения

и плавное снижение в процессе обучения



Training

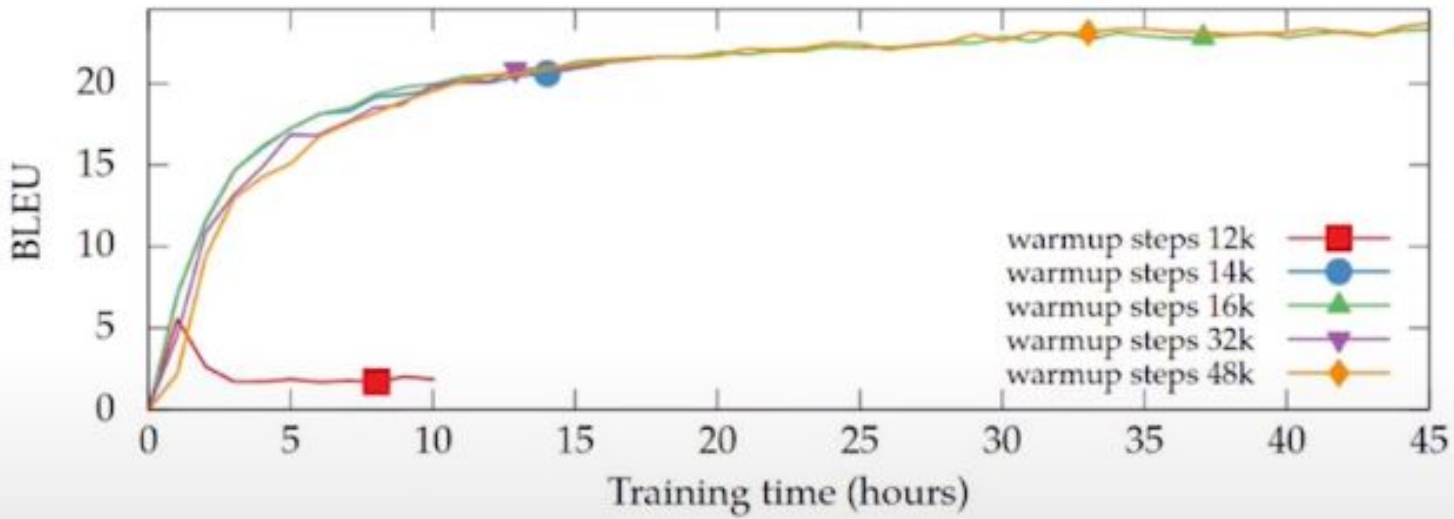
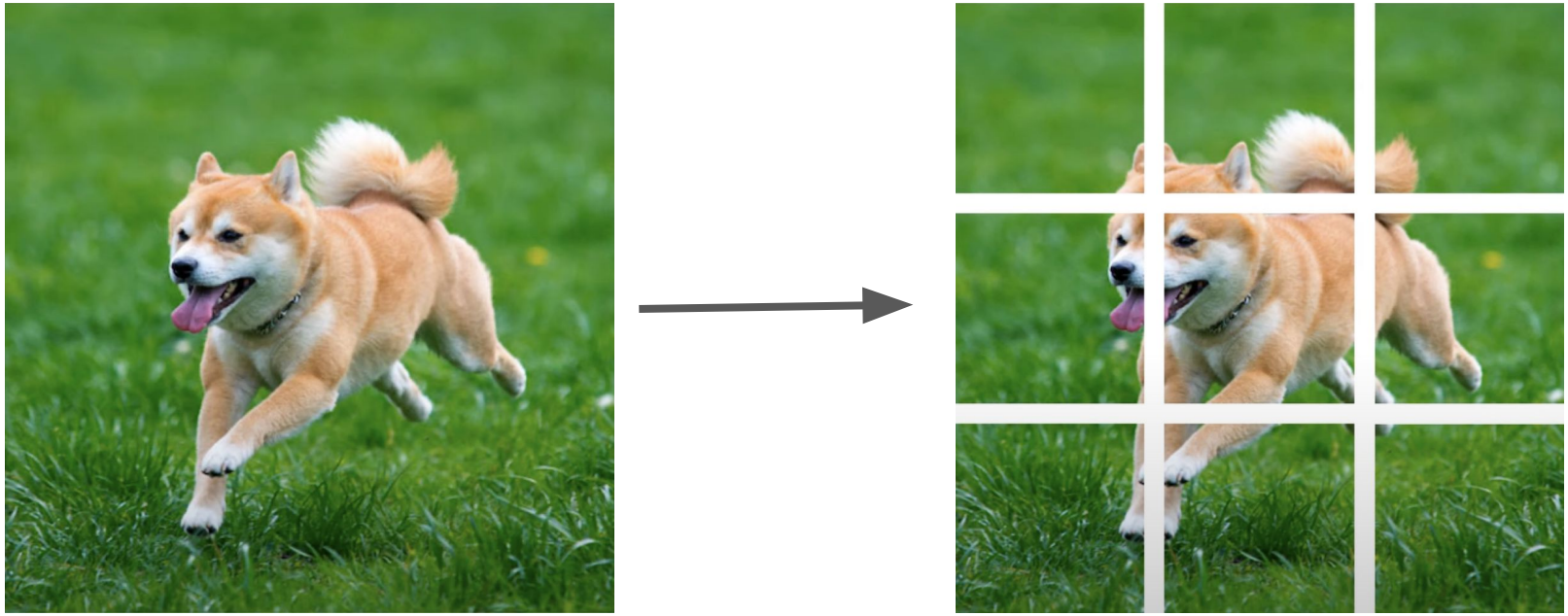
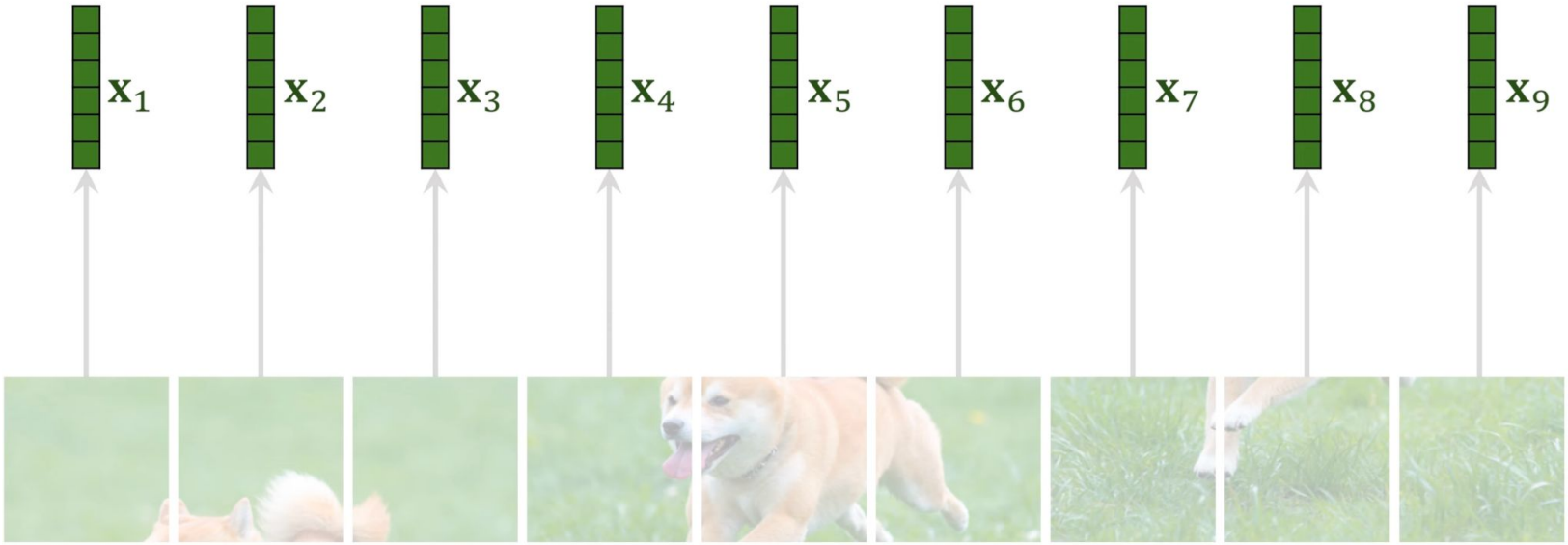
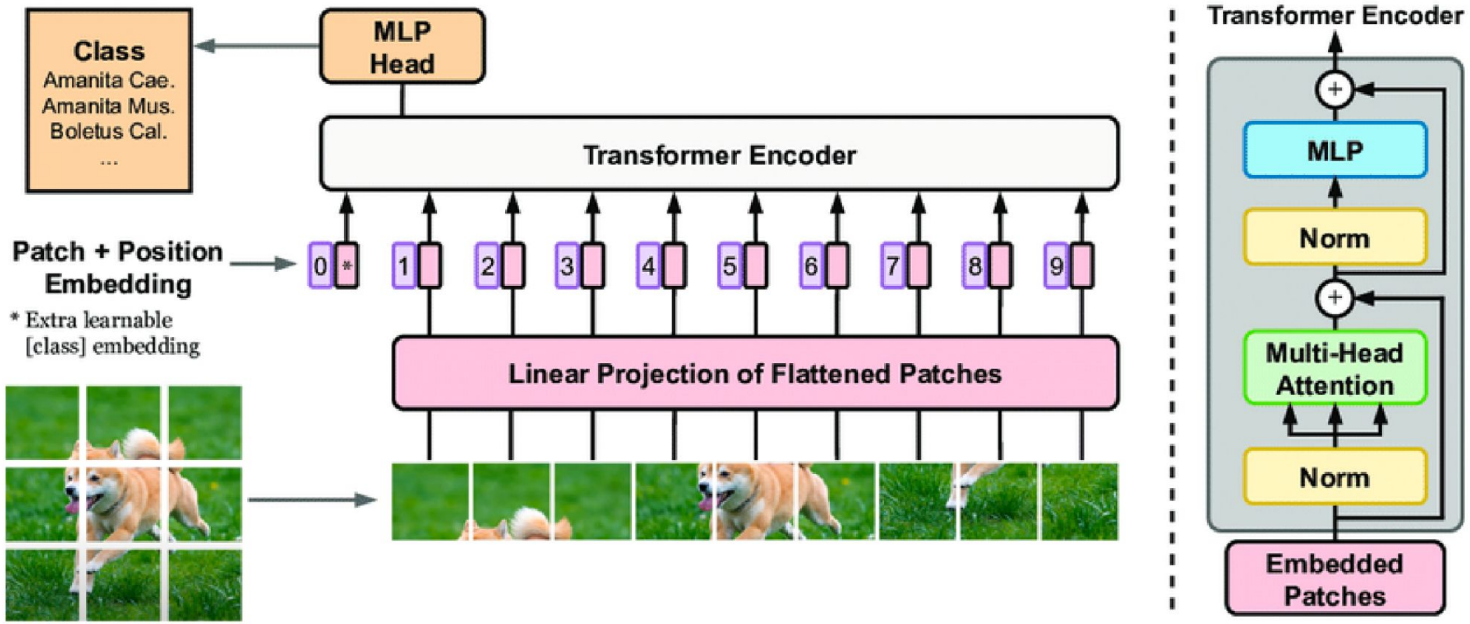


Figure 8: Effect of the warmup steps on a single GPU. All trained on CzEng 1.0 with the default batch size (1500) and learning rate (0.20).

Vision Transformer (ViT)



Vision Transformer (ViT)



Vision Transformer (ViT)